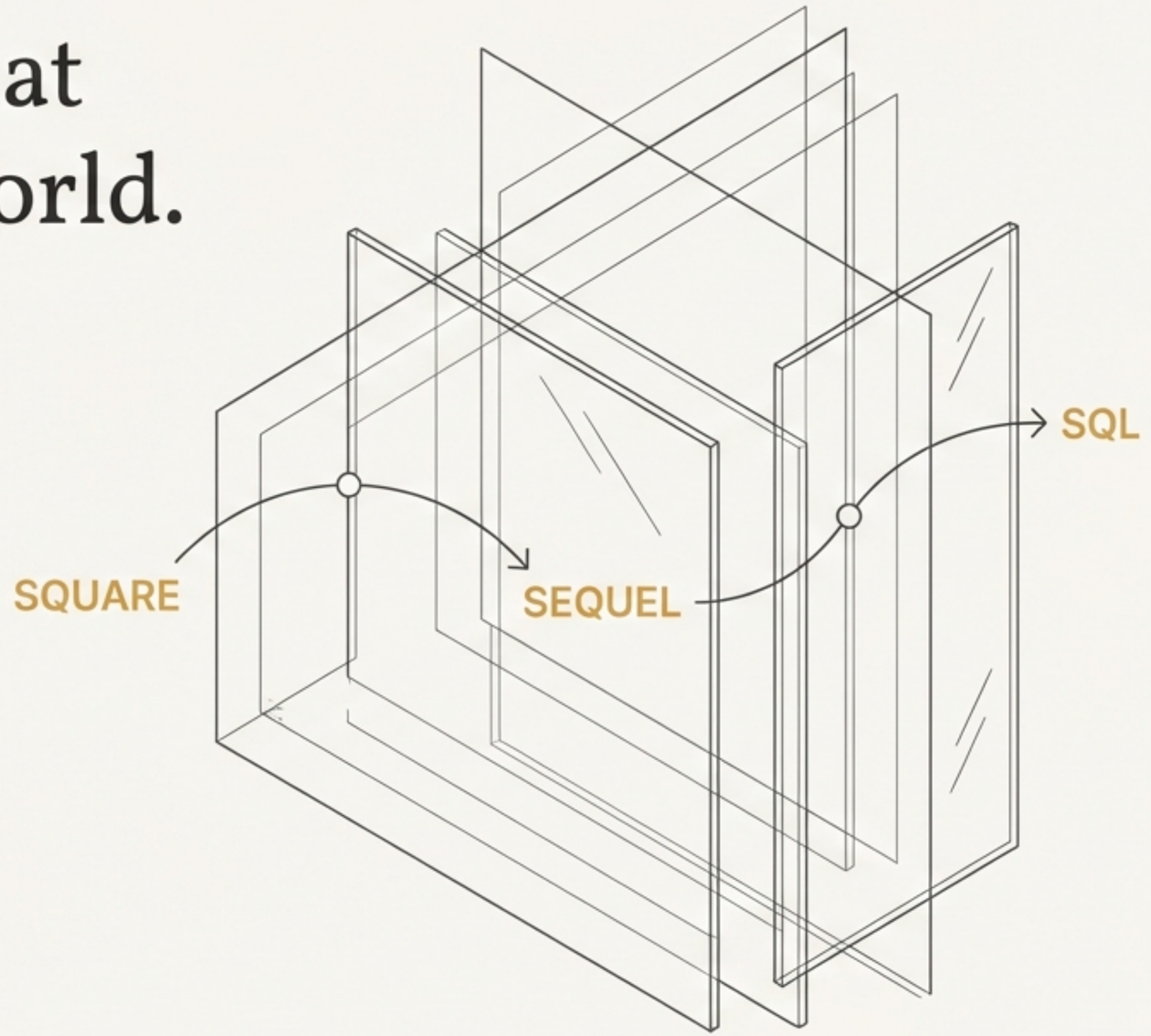
SQL: The Language That Built the Relational World.

Structured Query Language (SQL) is considered one of the major reasons for the commercial success of relational databases. Its origin stems from relational predicate calculus and an early language called SQUARE.

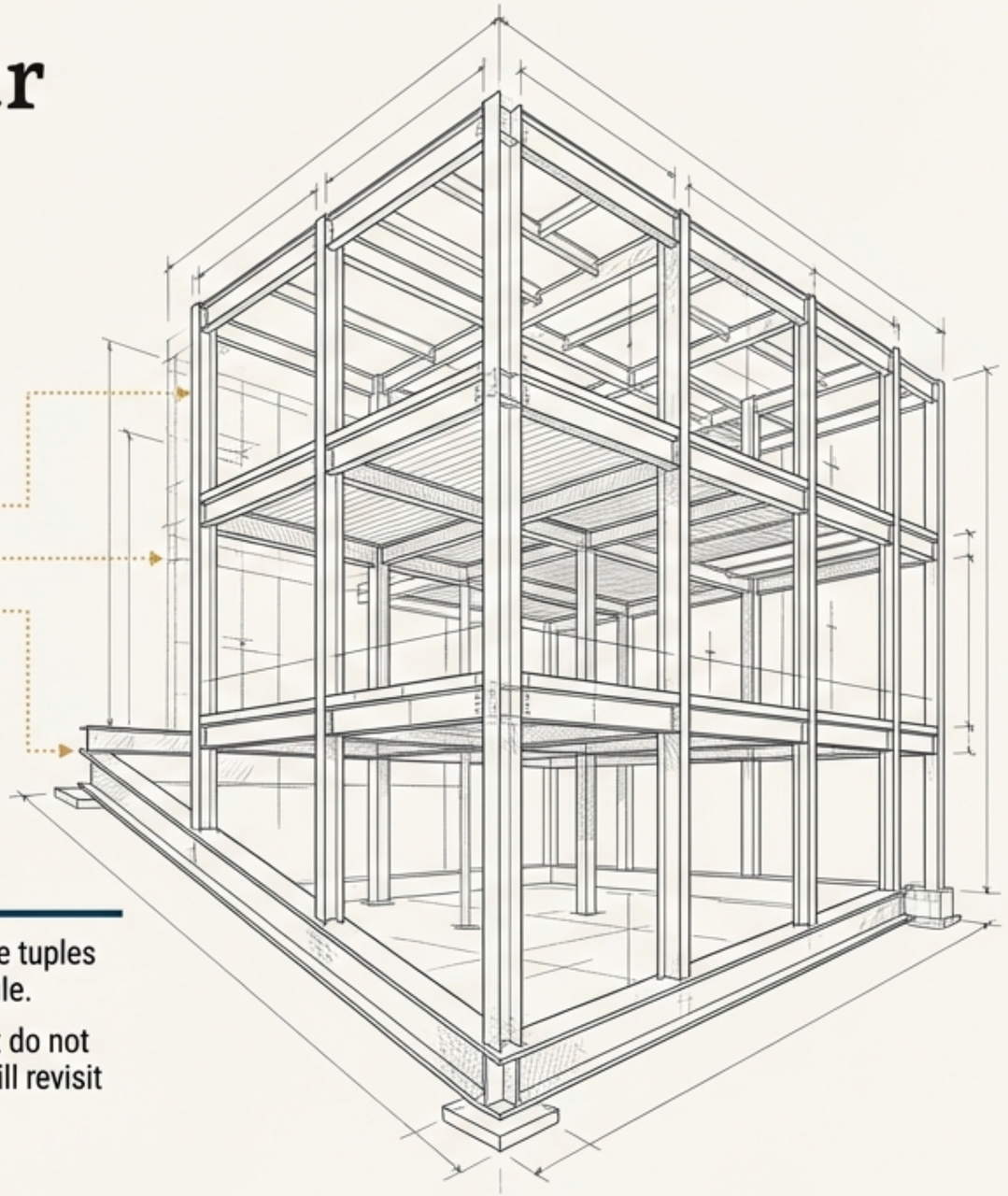
IBM's research led to the development of **'SEQUEL' (Structured English Query Language)**, later abbreviated to SQL for copyright reasons.



The Blueprint: Defining Your Data with CREATE TABLE.

The `CREATE TABLE` command is the starting point for building any database. It specifies a new relation (table), its attributes (columns), their data types, and initial constraints.

```
CREATE TABLE COMPANY.EMPLOYEE (
  Dname VARCHAR(15) NOT NULL,
  Dnumber INT NOT NULL,
  ...
);
```



Key Components

- **Table Name:** The unique identifier for the new relation (e.g., `EMPLOYEE`).
- **Attributes & Types:** A list of columns and their data types (e.g., `Dname VARCHAR(15)`).
- **Schema (Optional):** Specify which schema the table belongs to (e.g., `COMPANY.EMPLOYEE`).

Core Concepts

- **Base Tables:** Physical relations whose tuples are actually created and stored as a file.
- **Virtual Tables (Views):** Relations that do not correspond to any physical file. We will revisit these.

Enforcing the Rules: Core Integrity Constraints.

SQL supports the three fundamental constraint types from the relational model to guarantee the structural integrity of your data.

Key Constraint

Ensures a primary key value is unique. Enforced via **PRIMARY KEY** and **UNIQUE** clauses.

```
Dname VARCHAR(15) UNIQUE;
```

Entity Integrity Constraint

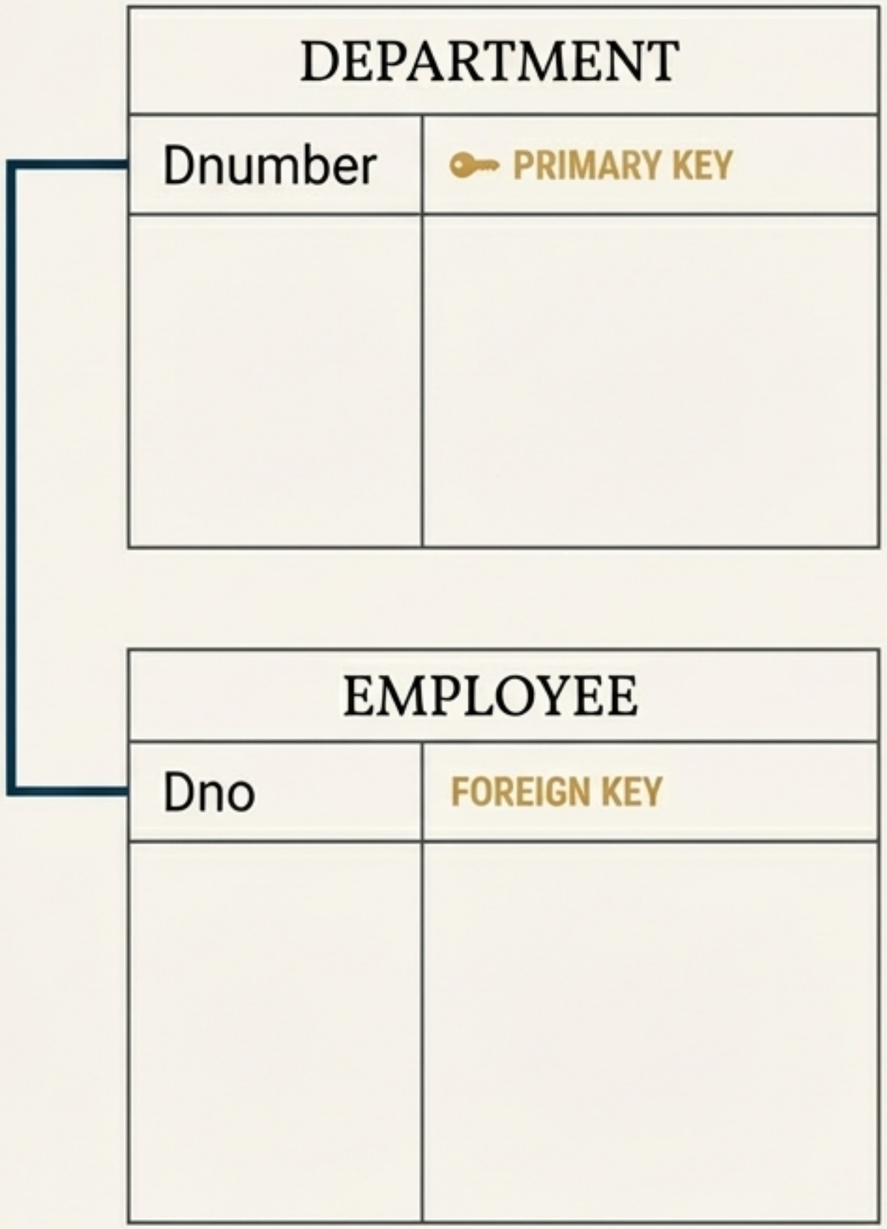
A primary key value cannot be **NULL**, ensuring every tuple has a unique, non-empty identifier.

```
Dnumber INT PRIMARY KEY;
```

This enforces both Key and Entity Integrity.

Referential Integrity Constraint

A foreign key value must either match an existing primary key value in another table or be **NULL**.



Finer Control: Specifying Attribute and Tuple Constraints

Beyond the core integrity rules, SQL allows for more specific constraints on attribute domains and tuple values.

Attribute-Level Constraints

- `NOT NULL`: Prohibits `NULL` values for a specific attribute.
- `DEFAULT <value>`: Assigns a default value if one isn't provided.
- `CHECK` (Attribute): Enforces a custom condition an attribute's value.

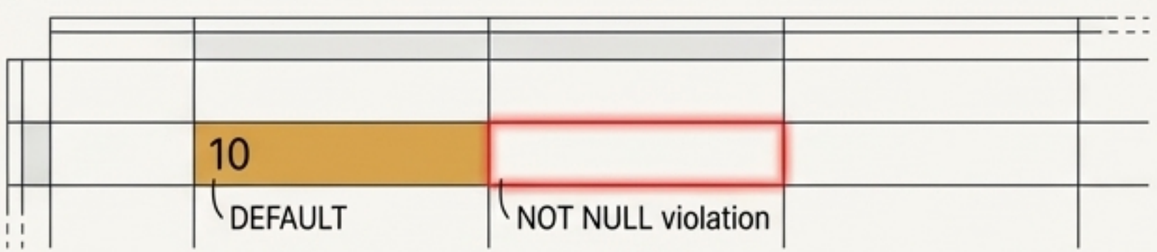
```
Dnumber INT NOT NULL
CHECK (Dnumber > 0 AND Dnumber < 21);
```

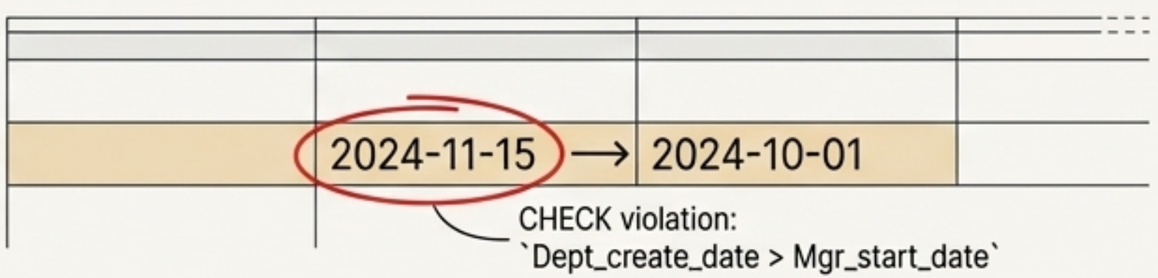
Tuple-Level & Referential Actions

- Tuple-Level `CHECK`: Enforces conditions across multiple attributes within a single tuple.

```
CHECK (Dept_create_date <= Mgr_start_date);
```

- Referential Actions: The `FOREIGN KEY` clause can include triggered actions (`ON DELETE` or `ON UPDATE`). Options: `SET NULL`, `CASCADE`, `SET DEFAULT`. The `CASCADE` option is particularly useful for relationship tables.





Interacting with Your Data: `INSERT`, `UPDATE`, `DELETE`.

Once tables are defined, these three Data Manipulation Language (DML) commands are used to modify the database content.

`INSERT`

Adds new tuples (rows) to a relation. Values must be listed in the specified column order. All constraints are automatically enforced.

```
INSERT INTO EMPLOYEE
VALUES (...);
```

`UPDATE`

Modifies attribute values of existing tuples. The **WHERE** clause selects the tuples to modify, and the **SET** clause specifies the new values.

```
UPDATE EMPLOYEE
SET SALARY = SALARY * 1.1
WHERE DNO = 5;
```

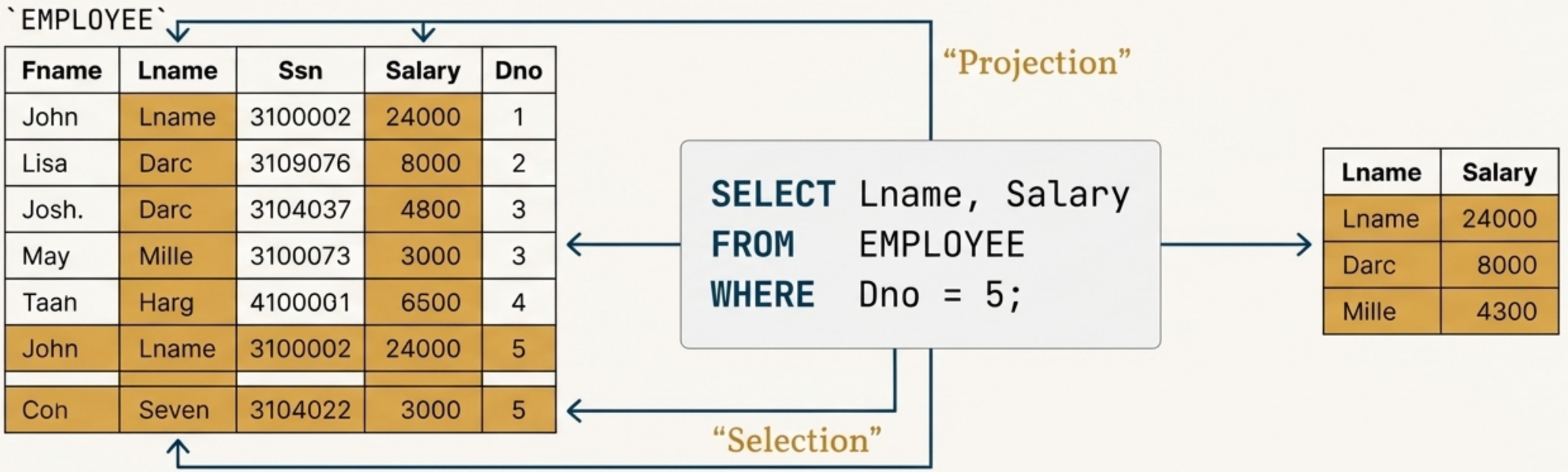
`DELETE`

Removes tuples from a relation. A **WHERE** clause selects which tuples to delete. Omitting it deletes all tuples. Referential actions are triggered.

```
DELETE FROM EMPLOYEE
WHERE Lname = 'Smith';
```

The Art of the Query: `SELECT-FROM-WHERE`.

The basic structure for retrieving information from the database, combining three fundamental clauses.



The Wildcard `*`

Use an asterisk (*) to retrieve all attribute values of the selected tuples, e.g., `SELECT * FROM EMPLOYEE;`.

Cross Product

A missing `WHERE` clause with multiple tables in the `FROM` clause results in all possible tuple combinations (Cartesian Product).

Refining Your Queries: Aliases, Distinctness, and Ordering.

Advanced techniques for cleaner, more precise data retrieval.

Managing Ambiguity with Aliases

When attribute names are duplicated across tables, qualify them with the relation name (e.g., `EMPLOYEE.Lname`). Aliases (or Tuple Variables) provide shorter names, improving readability and enabling self-joins.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

Eliminating Duplicates

By default, SQL results can include duplicate tuples. Use the **DISTINCT** keyword to ensure only unique rows remain.

Before

Lname	Salary
Smith	24000
Darc	8000
Smith	24000

DISTINCT →

After

Lname	Salary
Smith	24000
Darc	8000

Controlling Output Order

The **ORDER BY** clause sorts the final result. Use **ASC** for ascending (default) and **DESC** for descending.

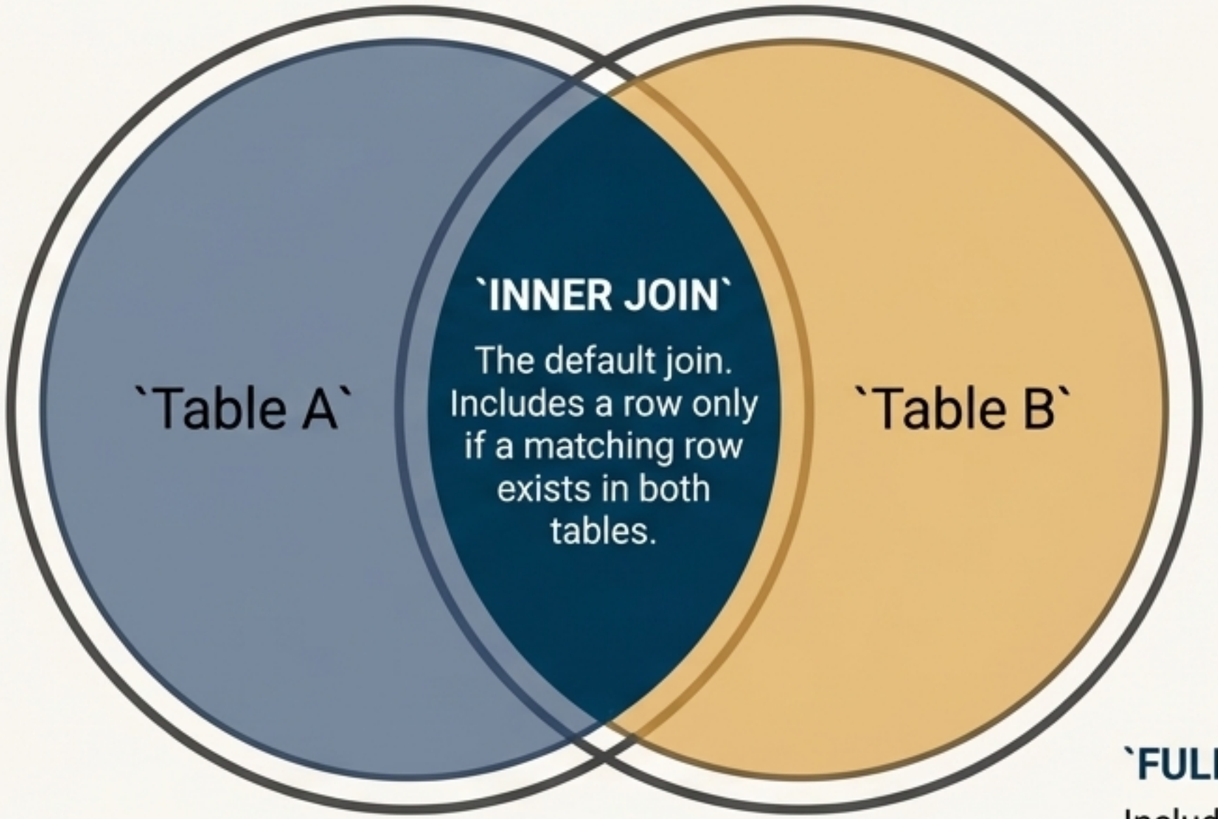
```
ORDER BY D.Dname DESC, E.Lname ASC;
```

Connecting the Dots: A Guide to `JOIN` Operations

Joins are fundamental to relational databases, allowing you to combine two or more tables in the `FROM` clause to create a new, unified result set.

`LEFT OUTER JOIN`

Includes every row from the *left* table. If there's no match in the right table, its columns are filled with `NULL`.



`RIGHT OUTER JOIN`

Includes every row from the *right* table. Unmatched rows from the left table get `NULL` values.

`FULL OUTER JOIN`

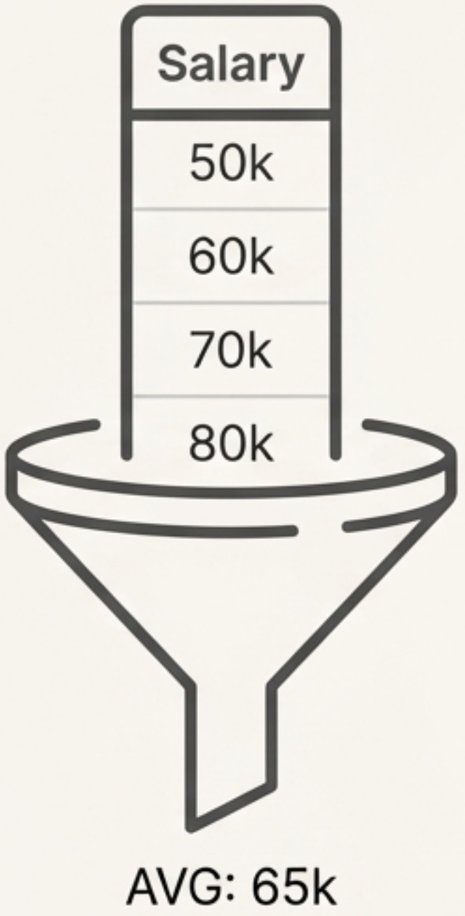
Includes all rows from both tables, filling in `NULL`s for non-matches on either side.

**`NATURAL JOIN`: A special type of `EQUIJOIN` where the join condition is implicitly created for all columns with the same name.

Seeing the Big Picture: Summarising Data with Aggregate Functions

Aggregate functions summarise information from multiple tuples into a single-tuple summary. NULL values are discarded when these functions are applied to a column.

- **COUNT()** Counts the number of rows. COUNT(*) counts all rows; COUNT(attribute) counts non-NULL values.
- **SUM()** Calculates the sum of a numeric column.
- **AVG()** Calculates the average of a numeric column.
- **MAX()** Returns the maximum value in a column.
- **MIN()** Returns the minimum value in a column.



Renaming Results

Use the **AS** keyword to give a meaningful name to the result of an aggregate function.

```
SELECT SUM(Salary) AS Total_Sal, AVG(Salary) AS Average_Sal
FROM EMPLOYEE;
```

Slicing the Data: Grouping and Filtering with GROUP BY and HAVING.

The GROUP BY Clause

Partitions a relation into subsets based on grouping attributes. Aggregate functions are then applied independently to each group.

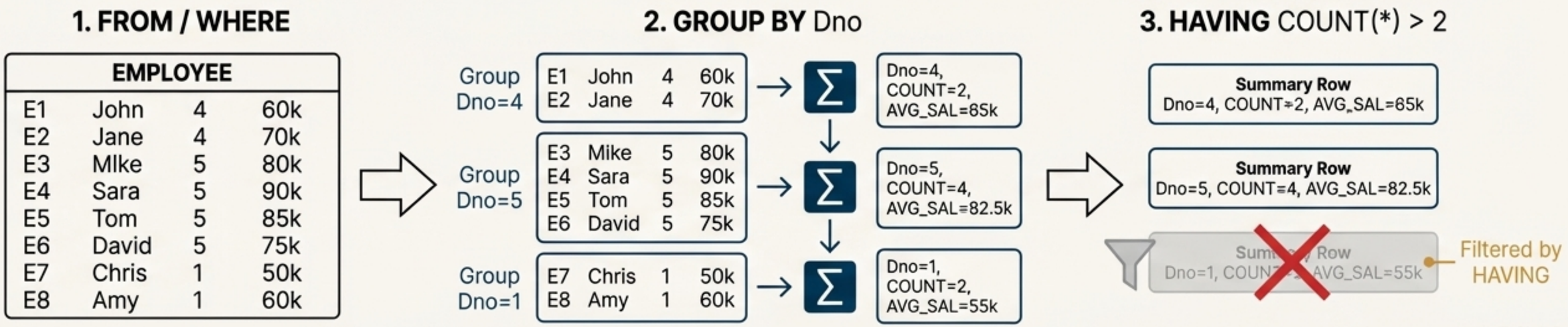
Rule: Any non-aggregate attribute in the **SELECT** clause must also be in the **GROUP BY** clause.

```
SELECT   Dno, COUNT(*), AVG(Salary)
FROM     EMPLOYEE
GROUP BY Dno;
```

The HAVING Clause

Provides a condition to filter entire groups after aggregation has occurred. **WHERE** filters individual rows before grouping; **HAVING** filters entire groups after.

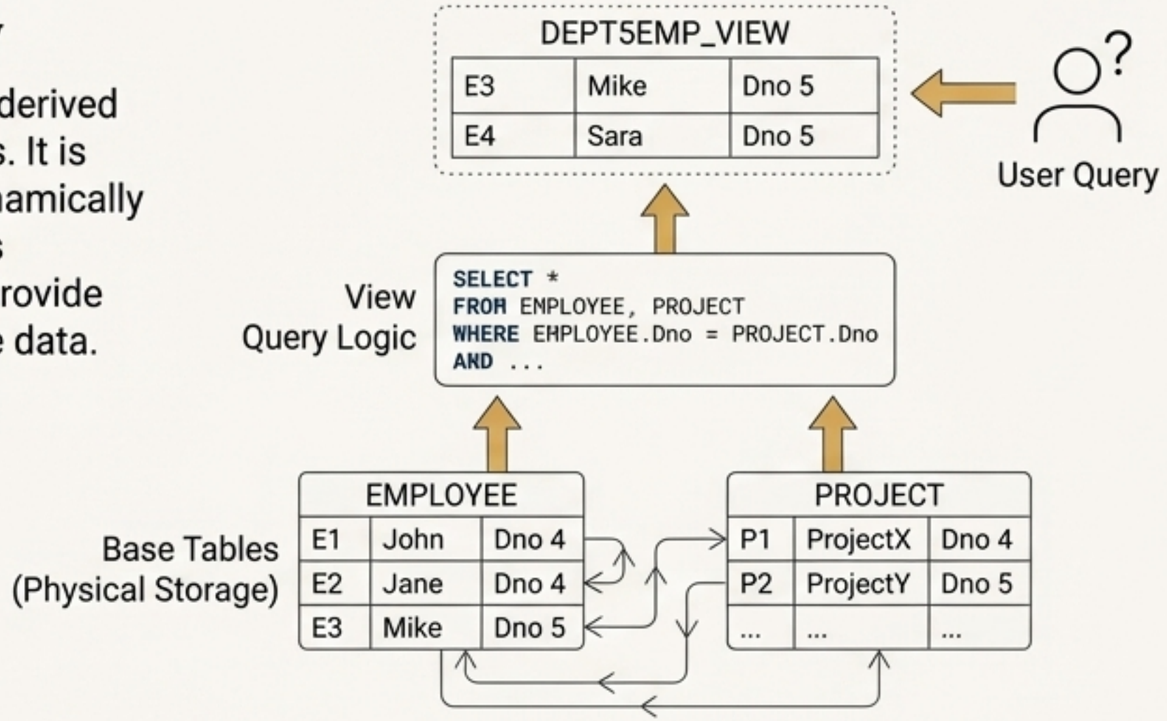
```
...
GROUP BY Pnumber, Pname
HAVING   COUNT(*) > 2;
```



Creating Custom Blueprints: Virtual Tables with CREATE VIEW

The Concept of a View

A View is a single, virtual table derived from other base tables or views. It is not physically stored, but is dynamically computed when queried. Views simplify complex queries and provide a stable, secure interface to the data.



Defining a View

Use the CREATE VIEW command, providing a name and the defining query.
Once created, a view can be used in the FROM clause just like a base table.

Views for Security

Views are a powerful authorization mechanism, used to hide certain attributes or tuples from users.

```
CREATE VIEW DEPT5EMP AS
SELECT * FROM EMPLOYEE WHERE Dno = 5;
```

Automating Reactions: Active Databases with `CREATE TRIGGER`.

Triggers specify automatic actions that the database system will perform when certain events and conditions occur. They are used to monitor the database and enforce complex business logic that goes beyond simple constraints.

The Three Components of a Trigger

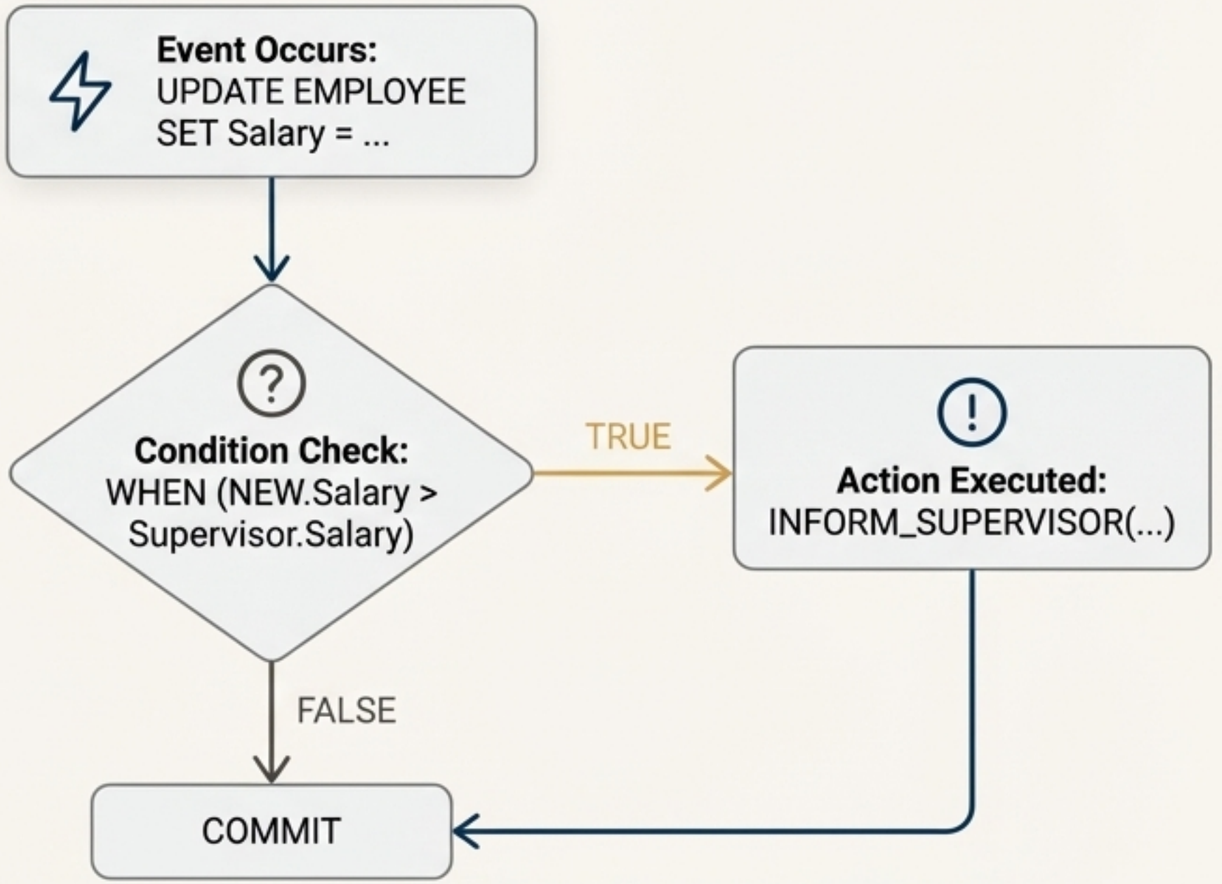
Event(s)
The action that activates the trigger (e.g., `INSERT`, `UPDATE`, `DELETE` on a specific table).

Condition
A `WHEN` clause that is evaluated. If true, the action is executed.

Action
The procedure or SQL statements to be executed.

Use Case Example

A trigger could check if a new employee's salary is greater than their supervisor's before an `INSERT` is committed, preventing invalid data entry.



Renovations and Extensions: Evolving Your Schema

Schema evolution commands allow a DBA to change the schema while the database is operational, without requiring a full recompilation.

The `ALTER TABLE` Command

Modifies the structure of an existing table.

- **ADD COLUMN**: Adds a new attribute.
- **DROP COLUMN**: Removes an attribute.
- **ALTER COLUMN**: Changes a column's definition.
- **ADD/DROP CONSTRAINT**: Adds or removes a constraint.

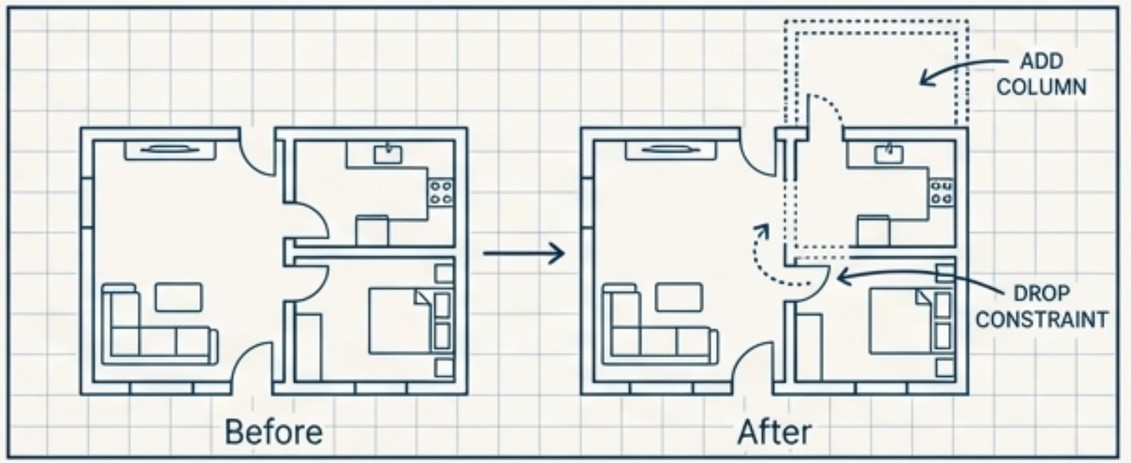
```
ALTER TABLE EMPLOYEE ADD COLUMN Job VARCHAR(12);
```

The `DROP` Command

Used to remove named schema elements entirely, such as tables or views.

- **RESTRICT**: Fails if other objects depend on the element.
- **CASCADE**: Also removes any dependent objects.

```
DROP SCHEMA COMPANY CASCADE;
```



The Challenge of the Unknown: Handling `NULL` Values

The Meanings of `NULL`

`NULL` represents an “unknown value.” It can also mean “unavailable” or “not applicable.” Each `NULL` is considered distinct from every other `NULL`.

Querying for `NULL`

You cannot use standard operators like = `NULL`. Instead, you must use the specific operators: **`IS NULL`** or **`IS NOT NULL`**.

Three-Valued Logic

SQL evaluates conditions involving `NULL` using a logic system with three outcomes: `TRUE`, `FALSE`, and `UNKNOWN`. A tuple is included in a result only if the `WHERE` clause evaluates to `TRUE`.

A	B	A AND B	A OR B
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

The Complete SQL Query Structure

This is the expanded block structure for a standard SQL retrieval query. Understanding this conceptual processing order is key to mastering complex queries.